



周志华《机器学习》西瓜书 手推笔记（V3.1）

第五章 《神经网络》

作者：王博（Kings）、Sophia

博士微信：**Kingsplus** （添加时请备注 学校/单位+专业）

Github: <https://github.com/Sophia-11/Machine-Learning-Notes>

（荣登趋势榜），QQ 交流群：**1072190864**

公众号【计算机视觉联盟】持续更新

后台回复【**西瓜书手推笔记**】可下载 pdf 打印版本



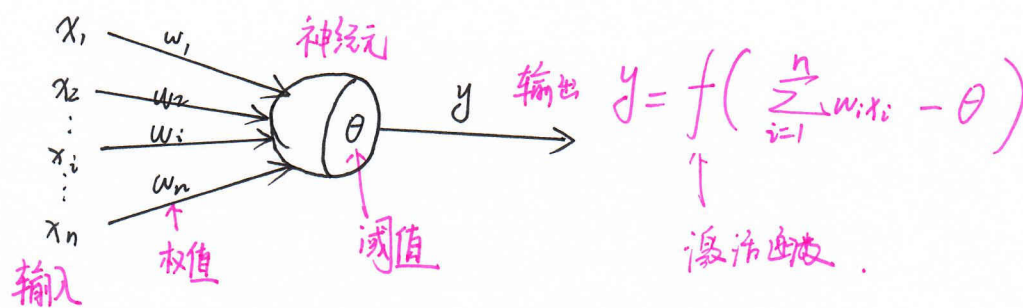
已完结待更笔记：《深度学习-花书手推笔记》、《无人驾驶手推笔记》、《SLAM 十四讲》
请继续关注公众号【计算机视觉联盟】最新消息或 Github

5.1 神经元模型

机器学习中谈论神经网络指“神经网络学习”

神经网络基本成分是神经元 (neuron) 模型 (unit).

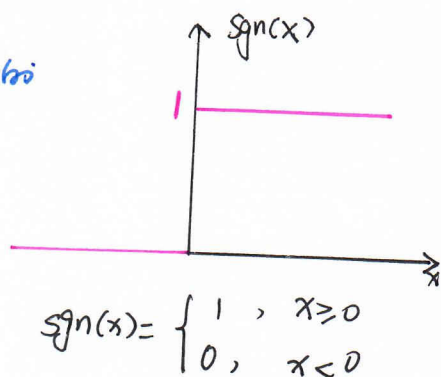
1943年, McCulloch and Pitts : M-P 神经元模型



M-P 神经元模型

激活函数

理想的

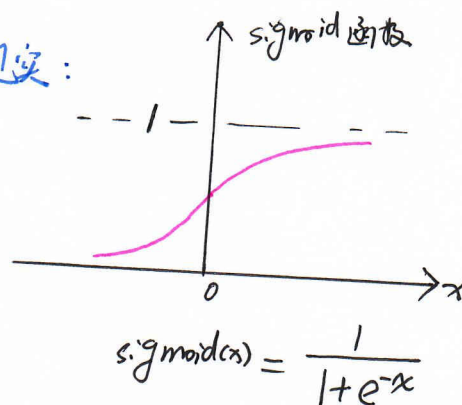


阶跃函数

要么激活, 1; 要么抑制, 0

但不连续 不光滑

现实:



Sigmoid 函数

0~1 之间

可微 连续

把许多这样神经元按一定层次连接就得到神经网络

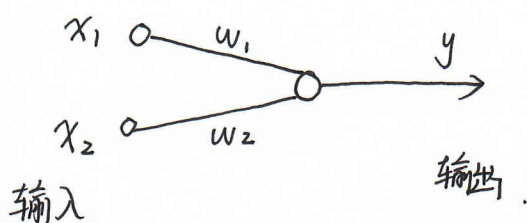
第5章 神经网络

5.2 感知机与多层网络



threshold logic unit

Perceptron 感知机由两层神经元组成，又称“阈值逻辑单元”



公众号
【计算机视觉联盟】

感知机可实现与、或、非运算， $y = f(\sum w_i x_i - \theta)$

“与运算” $x_1 \wedge x_2$

f 为阶跃函数 $\text{sgn}(x) = \begin{cases} 1, & x \geq 0 \\ 0, & x < 0 \end{cases}$

$$w_1 = w_2 = 1, \theta = 2 \quad \text{则} \quad y = f(1 \cdot x_1 + 1 \cdot x_2 - 2)$$

仅在 $x_1 = x_2 = 1$ 时 $y = 1$

“或运算” $x_1 \vee x_2$

$$w_1 = w_2 = 1, \theta = 0.5 \quad \text{则} \quad y = f(1 \cdot x_1 + 1 \cdot x_2 - 0.5)$$

当 $x_1 = 1$ 或 $x_2 = 1$ 时 $y = 1$

“非运算” $\neg x_1$

$$w_1 = -0.6, w_2 = 0, \theta = -0.5 \quad \text{则} \quad y = f(-0.6 \cdot x_1 + 0.5)$$

当 $x_1 = 1$ 时, $y = 0$, $x_1 = 0$ 时 $y = 1$

更一般的，给定数据集，权重 $w_i (i=1, 2, \dots, n)$ 阈值 θ 可通过学习得到

θ 可看做 w_{n+1} ，输入永远是 -1。

若当前感知机输出为 $\hat{y}$ ，感知机权重调整

$$w_i \leftarrow w_i + \Delta w_i \quad (5.1)$$

$$\Delta w_i = \eta (y - \hat{y}) x_i \quad (5.2)$$

↑
学习率 $\eta \in (0, 1)$

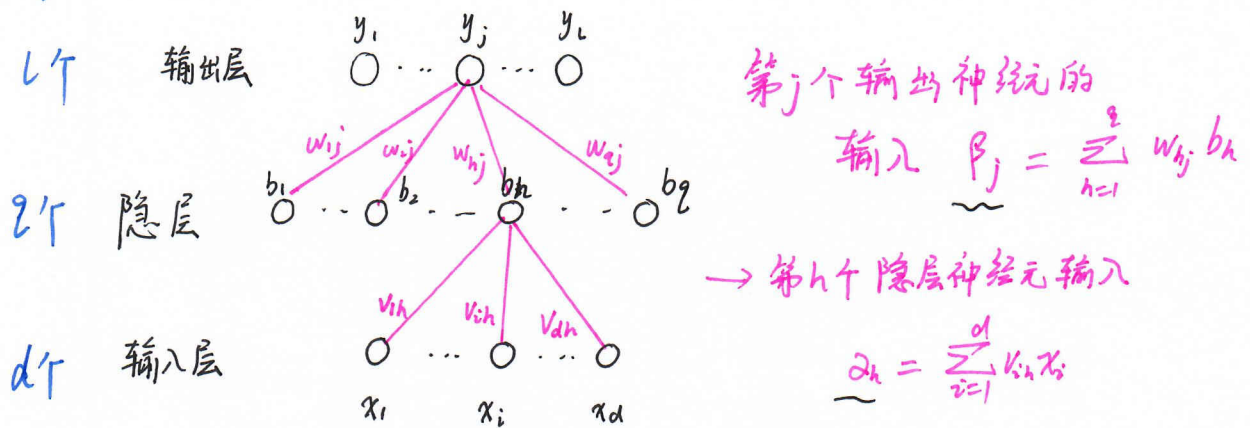
单层感知机只能“与”“或”“非”，非线性“异或”解决不了

两层感知机可解决“异或”引出多层前馈神经网络

5.3 误差逆传播算法

error Back Propagation BP 是其中杰出代表

通常说“BP网络”一般指用BP算法训练多层前馈神经网络



训练例 (x_k, y_k) 假定输出为 $\hat{y}_k = (\hat{y}_1^k, \hat{y}_2^k, \dots, \hat{y}_L^k)$

即
$$\hat{y}_j^k = f(\beta_j - \theta_j) \quad (5.3)$$

网络上的均方误差为
$$E_k = \frac{1}{2} \sum_{j=1}^L (\hat{y}_j^k - y_j^k)^2 \quad (5.4)$$

需要多少个参数呢？
 $d \times q$ 个权值 $q \times L$ 个权值
 q 个阈值 L 个阈值

$$d \times q + q \times L + q + L = (d + L + 1)q + L \text{ 个阈值}$$

任意参数更新公式为
$$v \leftarrow v + \Delta v \quad (5.5)$$

BP算法基于梯度下降策略，以目标负梯度方向对参数调整。

以 w_{hj} 为例：

$$\Delta w_{hj} = -\eta \frac{\partial E_k}{\partial w_{hj}} \quad (5.6)$$

$$\frac{\partial E_k}{\partial w_{hj}} = \underbrace{\frac{\partial E_k}{\partial \hat{y}_j^k}}_{\text{最终影响 } E_k} \cdot \underbrace{\frac{\partial \hat{y}_j^k}{\partial \beta_j}}_{\text{再影响 } \hat{y}_j^k} \cdot \underbrace{\frac{\partial \beta_j}{\partial w_{hj}}}_{\text{先影响 } \beta_j} \quad (5.7)$$

5.3 误差逆传播算法

$$\frac{\partial E_k}{\partial w_{hj}} = \frac{\partial E_k}{\partial \hat{y}_j^k} \cdot \frac{\partial \hat{y}_j^k}{\partial \beta_j} \cdot \frac{\partial \beta_j}{\partial w_{hj}} \quad (5.7)$$

(5.9) $f'(x) = f(x)(1-f(x))$ (5.3) (5.4)

$$\frac{\partial \beta_j}{\partial w_{hj}} = b_h \quad (5.8)$$

$$\begin{aligned} g_j &= - \frac{\partial E_k}{\partial \hat{y}_j^k} \cdot \frac{\partial \hat{y}_j^k}{\partial \beta_j} = -(\hat{y}_j^k - y_j^k) f'(\beta_j - \theta_j) \\ &= \hat{y}_j^k (1 - \hat{y}_j^k) (y_j^k - \hat{y}_j^k) \quad (5.10) \end{aligned}$$

公众号
【计算机视觉联盟】

合并 (5.10) (5.8) $\Rightarrow$ (5.7) $\Rightarrow$ (5.6)

$$\Delta w_{hj} = -\eta \frac{\partial E_k}{\partial w_{hj}} = \eta g_j b_h \quad (5.11)$$

同理 $\left\{ \begin{array}{l} \Delta \theta_j = -\eta g_j \quad (5.12) \\ \Delta v_{ih} = \eta e_h x_i \quad (5.13) \\ \Delta v_h = -\eta e_h \quad (5.14) \end{array} \right\}$ 中的 e_h

$$\begin{aligned} e_h &= - \frac{\partial E_k}{\partial b_h} \cdot \frac{\partial b_h}{\partial z_h} \\ &= - \sum_{j=1}^L \frac{\partial E_k}{\partial \beta_j} \cdot \frac{\partial \beta_j}{\partial b_h} f'(z_h - \gamma_h) \\ &= \sum_{j=1}^L w_{hj} g_j f'(z_h - \gamma_h) \\ &= b_h (1 - b_h) \sum_{j=1}^L w_{hj} g_j \quad (5.15) \end{aligned}$$

BP目标是最小化训练集D上的累积误差

$$E = \frac{1}{m} \sum_{k=1}^m E_k \quad (5.16)$$

如何缓解BP网络过拟合?

(1) “早停”: 数据分为训练集和验证集, 训练集用于计算梯度、更新权、阈值。
验证集来估计误差, 若训练集误差降低但验证集升高, 则停止训练。

(2) “正则化”: 在误差目标函数上增加一个可用于描述网络复杂度部分, 如权与阈值的平方。

$$E = \lambda \frac{1}{m} \sum_{k=1}^m E_k + (1-\lambda) \sum_i w_i^2$$

其中 $\lambda \in (0, 1)$ 用于对训练误差和网络复杂度两项折中

5.4 全局最小与局部极小

↓
global minimum

↓
local minimum

公众号
【计算机视觉联盟】

对 w^* 和 θ^* , 若存在 $\varepsilon > 0$ 使得

$$\forall (w; \theta) \in \{(w; \theta) \mid \|(w; \theta) - (w^*; \theta^*)\| \leq \varepsilon\}$$

都有 $E(w; \theta) \geq E(w^*; \theta^*)$ 成立, 则 $(w^*; \theta^*)$ 为局部极小解.

若任意 $(w; \theta)$ 都有 $E(w; \theta) \geq E(w^*; \theta)$, 则为全局最小解.

人们常采用以下策略来试图“跳出”局部极小, 从而进一步接近全局最小:

- (1) 以多组不同参数值初始化多个神经网络, 按标准方法训练后, 取其中误差最小的解作为最终参数.

相当于从多个不同的初始点开始搜索, 陷入不同的局部极小, 从而选择有可能获得更接近全局最小的结果.

- (2) 使用“模拟退火”技术. (simulated annealing)

模拟退火每一步都以一定概率接受比当前解更差的结果.

每步迭代中, 接受“次优解”的概率要随时间推移而降低, 保证算法稳定.

- (3) 使用随机梯度下降

在计算梯度时加入了随机因素.

即便陷入局部极小点, 计算的梯度仍可能不为零.

遗传算法 (genetic algorithms) 也常用来训练神经网络以更好地逼近全局最小

上述方法理论上不够, 启发式

5.5 其他常见神经网络

5.5.1 RBF网络

RBF (Radial Basis Function, 径向基函数) 网络是一种单隐层前馈神经网络, 使用径向基函数作为隐层神经元激活函数. 输出层则是对隐层神经元输出的线性组合.

假定输入为 d 维向量 x , 输出为实值.

RBF可表示为:

$$\varphi(x) = \sum_{i=1}^q w_i p(x, c_i) \quad (5.18)$$

$\swarrow$ 权重 $\searrow$ 中心
 $\downarrow$
 径向基函数
 $\downarrow$ 定义

某种沿径向对称的标量函数, 通常定义为样本 x 到数据中心 c_i 之间的欧氏距离的单调函数.

高斯径向基函数形如:

$$p(x, c_i) = e^{-\beta_i \|x - c_i\|^2} \quad (5.19)$$

具有足够多隐层神经元的RBF网络能以任意精度逼近^{任意}连续函数.

两步: ① 确定神经元中心 c_i . 随机采样. 聚类

② 利用BP确定 w_i 和 β_i .

5.5.2 ART网络

什么是竞争型学习?

竞争型学习 (competitive learning) 是神经网络一种常用的无监督学习策略. 输出神经元相互竞争, 每一时刻仅有一个竞争获得胜利. 该神经元激活, 其它被抑制.

这种称为“胜者通吃” (winner-take-all) / 胜者

5.5 其他常见神经网络

5.5.2 ART网络

ART (Adaptive Resonance Theory, 自适应谐振网络理论) 是竞争性学习代表

网络由 比较层、识别层、识别阈值和重置模块构成。

↓
接收输入样本

↓
每个神经元对应一个模式类, 神经元数目可在训练过程中动态增长。
增加新的模式类

竞争最简单的方式

1. 计算输入向量与每个识别神经元所对应的模式类的代表向量之间的距离
距离最小者胜
2. 获胜神经元向其他识别层神经元发送信号, 抑制激活
3. 输入与获胜神经元的相似度大于阈值, 归类该属性
4. 更新连接权重, 后期相似样本计算更大相似度
5. 若相似度小于阈值, 增设一个新的神经元, 当前输入为代表向量。

识别阈值

阈值高, 输入样本会被细分为多种类别

阈值低, 产生比较少, 比较粗略的类

stability - plasticity dilemma

ART比较好地缓解了竞争型学习中的“可塑性 - 稳定性窘境”。

可塑性指有学习新知识的能力

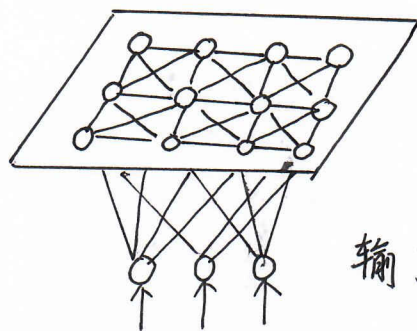
稳定性指学习新知识时要保持对旧知识记忆。

于是优点: 可进行增量学习 或 在线学习。
incremental learning online learning

5.5 其他常见神经网络

5.5.3 SOM 网络 (Self-Organizing Map)

SOM 网络是一种 **竞争学习型** 的 **无监督神经网络**，能将高维映射到低维，同时保持高维拓扑结构。高维相似点映射为输出层邻近神经元。



输出层：神经元以二维阵列排列二维空间，
每个神经元都有一个权向量。

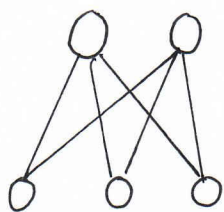
输入层

步骤：接收样本，每个输出层神经元会计算该样本与自身携带的权向量的距离。距离最近获胜，称最佳匹配单元。

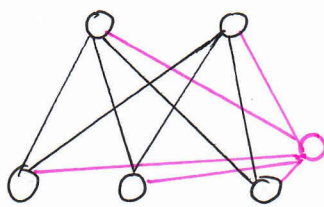
周围神经元权向量调整，使得权向量与当前输入样本距离缩小。不断迭代，直至收敛。

5.5.4 级联相关网络 (Cascade - Correlation)

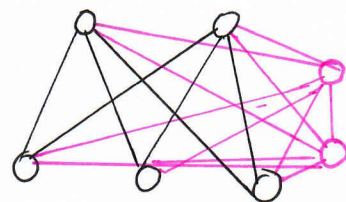
结构自适应网络 将网络结构也作为学习目标，希望找到合适网络结构。
重要代表。



初始



增加一个隐层结点



增加第二个隐层结点

级联：建立层次连接的结构。开始时只有输入输出层；随着训练新的隐层加入。

与一般前馈神经网络相比，级联相关网络无需设置网络层次、隐层神经元数目，且训练速度快。但数据较少时容易过拟合。

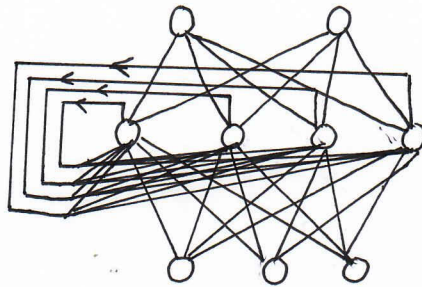
5.5 其他常见神经网络

5.5.5 Elman 网络

公众号
【计算机视觉联盟】

“递归神经网络” (recurrent neural networks) 允许出现环形结构，从而让一些神经元的输出反馈回来作为输入信号。

使得 t 时刻输出状态不仅与 t 时刻输入有关，还与 $t-1$ 时刻网络状态有关，从而能处理与时间有关的动态变化。



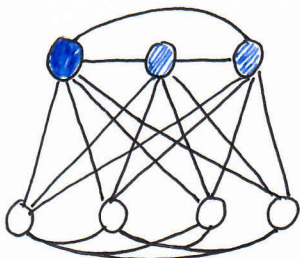
Elman 网络结构

与多层前馈网络相似，但隐层神经元输出被反馈回来，与下一时刻输入一起作为下一次隐层输入。 sigmoid 激活函数。 BP 算法。

5.5.6 Boltzmann 机

神经网络有一类模型是为网络定义一个“能量”，能量最小即为理想。训练就是为最小化此能量函数。

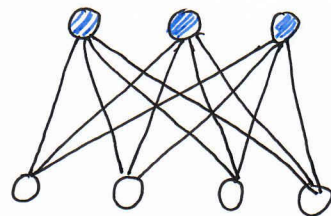
Boltzmann 机就是一种“基于能量的模型” (energy-based-model)



a) Boltzmann 机

隐层

显层



b) 受限 Boltzmann 机: RBM
常用

神经元分两层 { 显层：用于数据的输入和输出
隐层：数据的内在表达

5.5 其他常见神经网络

5.5.6 Boltzmann 机

Boltzmann 机中神经元都是布尔型，取 0, 1 状态
 $\uparrow$ 抑制 $\downarrow$ 激活

令向量 $s \in \{0, 1\}^n$ 表 n 个神经元状态， w_{ij} 表神经元 i, j 权。

θ_i 表神经元 i 的阈值。状态向量 s 所对应 Boltzmann 机能量定义为：

$$E(s) = - \sum_{i=1}^{n-1} \sum_{j=i+1}^n w_{ij} s_i s_j - \sum_{i=1}^n \theta_i s_i \quad (5.20)$$

若神经元以任意不依赖输入层顺序更新，则最终将达到 Boltzmann 分布。此时状态向量 s 出现的概率将仅由其能量所有可能状态向量的能量确定：

$$P(s) = \frac{e^{-E(s)}}{\sum_t e^{-E(t)}} \quad (5.21)$$

训练过程：每个样本视为一个状态向量，使其出现概率尽可能大。

标准 Boltzmann 机复杂度高，现实常采用受限 Boltzmann 机 (RBM)

RBM 常用“对比散度” (Contrastive Divergence, CD) 进行训练。

假设 d 个显层神经元

q 个隐层神经元

v 和 h 分别表显层与隐层状态向量。

$$P(v|h) = \prod_{i=1}^d P(v_i|h) \quad (5.22)$$

$$P(h|v) = \prod_{j=1}^q P(h_j|v) \quad (5.23)$$

每个样本：先根据 (5.23) 算隐层神经元状态分布概率

$\Downarrow$
根据概率分布采样得 h ；

$\Downarrow$
然后从 (5.22) 从 h 产生 v'

$v' \neq v$

权更新公式：

$$\Delta w = \eta (v h^T - v' h'^T) \quad (5.24)$$

5.6 深度学习

- (1) 典型的深度学习模型就是很深层的神经网络，增加隐层数目
- (2) 然而，多隐层神经网络难以直接用经典算法（如BP）进行训练，因为误差在多隐层内逆传播时，往往会“发散”（diverge）而不能收敛。
- (3) 无监督逐层训练是多隐层网络训练的有效手段。
(unsupervised layer-wise training)
训练时将上一层隐结点的输出作为输入，而本层隐结点的输出作为下一层隐结点的输入，称为“预训练”（pre-training）；预训练完成后，对整个网络“微调”。
- (4) 深度置信网络（deep belief network, DBN）（Hinton 2006）每层都是一个受限Boltzmann
整个网络可视为若干个RBM堆叠而得，使用无监督逐层训练时，第一层RBM训练，然后第二层预训练，... 完成后，利用BP算法对整个网络训练。
- (5) “预训练+微调”可视为将大量参数分组，局部较优聚合为全局寻优，有效节省训练开销。
- (6) “权共享”可节省训练开销，让一组神经元使用相同的连接权。
此策略在CNN中发挥了重要作用
- (7) “深度学习”又为“特征学习”或“表示学习”
通过多层处理，逐渐将初始“低层”特征转化“高层”表示，用“简单模型”完成复杂的分类学习任务。